

LIVRET D'EXCELLENCE

TOUT-EN-UN

Algorithmes Avancés

Le Backtracking & Le Jeu du Taquin

Fiche 1 : Le Cours Complet

Fiche 2 : 40 Flashcards

Fiche 3 : Quiz & Mémento

mathbacai.com

Contents

1	Le Cours Complet	3
1.1	Introduction aux Algorithmes Avancés	3
1.2	Le Backtracking (Retour sur Trace)	3
1.2.1	Principe Fondamental	3
1.2.2	Modélisation Graphique du Backtracking	3
1.2.3	Application : Le Problème des 8 Dames	4
1.3	Le Jeu du Taquin (Puzzle)	7
1.3.1	Principe de base	7
1.3.2	Représentation Visuelle	7
1.3.3	Concepts Clés de Modélisation Informatique	7
2	Le Recueil de Questions-Réponses	8
3	Quiz d'Évaluation & Fiches Mémo	13
3.1	Quiz d'Évaluation (10 Questions Types Bac)	13
3.2	Le Tableau Récapitulatif d'Arborescence	15
3.3	Fiche Mémo : Les 5 Règles d'Or pour le Bac	15

CHAPITRE 1

Le Cours Complet

1.1 Introduction aux Algorithmes Avancés

Dans les chapitres précédents, nous avons étudié des problèmes dont la résolution repose sur des formules directes, des tris ou des approximations numériques. Cependant, de nombreux problèmes réels (aide à la décision, cryptographie, intelligence artificielle, planification) possèdent un espace de recherche tellement vaste qu'une exploration naïve ou exhaustive est impossible dans un temps raisonnable.

Les algorithmes avancés introduisent des concepts d'exploration méthodique d'arbres de choix, d'optimisation heuristique et d'évaluation intelligente pour surmonter cette complexité combinatoire.

1.2 Le Backtracking (Retour sur Trace)

1.2.1 Principe Fondamental

Le **Backtracking** est une méthode de résolution algorithmique récursive qui explore pas à pas un espace de recherche pour trouver toutes les solutions possibles (ou une solution optimale) à un problème sous contraintes.

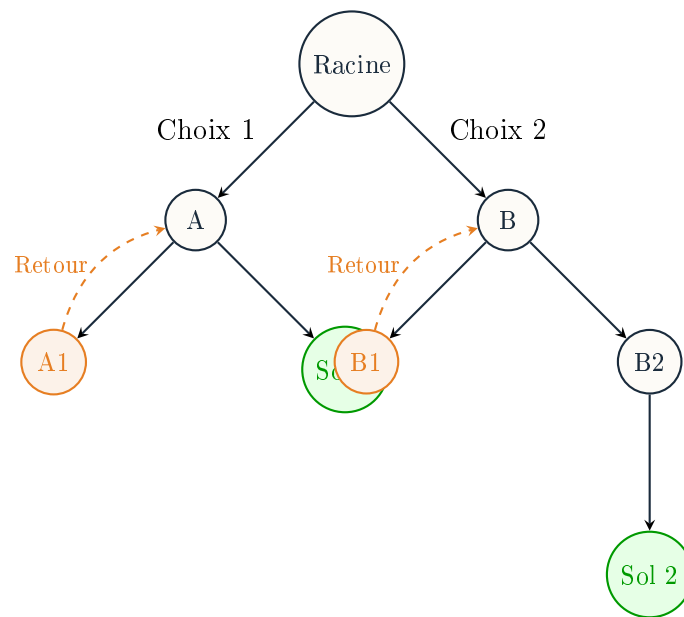
Le principe repose sur :

1. **Exploration progressive** : On avance en posant des choix successifs (décisions).
2. **Validation des contraintes** : À chaque étape, on vérifie si l'état intermédiaire respecte les règles du problème.
3. **Retour sur trace (Backtrack)** : Si une contrainte est violée ou qu'une impasse est atteinte, on annule la dernière décision prise et on revient à l'étape précédente pour tenter un autre choix.

Cette approche s'apparente à un parcours en profondeur (*Depth-First Search* ou DFS) d'un arbre représentant l'espace des états possibles, en coupant (élaguant) les branches qui violent les contraintes.

1.2.2 Modélisation Graphique du Backtracking

Le schéma suivant montre comment le cheminement progresse d'état en état, revenant en arrière en cas d'impasse.



1.2.3 Application : Le Problème des 8 Dames

Énoncé

Le problème consiste à placer 8 dames (reines) d'échecs sur un échiquier standard de 8×8 de manière à ce qu'aucune dame ne puisse en menacer une autre. Deux dames se menacent si elles sont situées sur la même ligne, la même colonne ou la même diagonale.

Analyse de l'espace de recherche

- Une approche exhaustive naïve consisterait à tester toutes les combinaisons possibles de placement de 8 pièces sur 64 cases, soit $\binom{64}{8} = 4\,426\,165\,368$ possibilités.
- En exploitant la contrainte qu'il doit y avoir **exactement une dame par colonne**, on réduit l'espace à $8^8 = 16\,777\,216$ configurations.
- Avec le **Backtracking**, l'arbre de recherche est élagué très tôt (dès qu'un conflit apparaît), ce qui permet de trouver les solutions en explorant seulement quelques milliers d'états.

Détection mathématique des diagonales

Soit une case de coordonnées (L, C) où L est la ligne (1 à 8) et C la colonne (1 à 8) :

- Deux cases de coordonnées (L_1, C_1) et (L_2, C_2) sont sur la même **diagonale montante** si :

$$L_1 + C_1 = L_2 + C_2$$

- Elles sont sur la même **diagonale descendante** si :

$$L_1 - C_1 = L_2 - C_2 \quad \text{ou de façon équivalente} \quad |L_1 - L_2| = |C_1 - C_2|$$

Implémentation Algorithmique et Pascal

Nous allons déclarer un tableau à une dimension pour représenter l'échiquier : `t[1..8]` de **Entiers**. L'indice représente le numéro de colonne, et la valeur de la case représente le numéro de ligne de la dame placée. Par exemple, si `t[3] = 5`, cela signifie qu'à la colonne 3, la dame se situe sur la ligne 5.

Algorithme de validation du placement

```

Fonction Valide(l, c : Entier) : Booleen
Var i : Entier, ok : Booleen
Début
    ok ← Vrai
    i ← 1
    Tant Que (i < c) ET ok Faire
        Si (t[i] = l) OU ( $\text{Abs}(t[i] - l) = \text{Abs}(i - c)$ ) Alors
            ok ← Faux
        FinSi
        i ← i + 1
    FinTantQue
    Retourner ok
Fin

```

Procédure Récursive de Backtracking

```

Procédure PlacerDames(c : Entier)
Var l : Entier
Début
    Si c > 8 Alors
        AfficherEchiquier() { Solution trouvée }
    Sinon
        Pour l De 1 À 8 Faire
            Si Valide(l, c) Alors
                t[c] ← l { Placement temporaire }
                PlacerDames(c + 1) { Appel récursif }
                { Le retour sur trace se fait automatiquement en reprenant la boucle Pour }
            FinSi
        FinPour
    FinSi
Fin

```

Traduction Pascal Complète

```

program Probleme8Dames;
uses crt;
type TabEchiquier = array[1..8] of integer;
var
    t : TabEchiquier;
    solCount : integer;

function Valide(l, c : integer) : boolean;
var i : integer; ok : boolean;
begin
    ok := true; i := 1;
    while (i < c) and ok do begin
        if (t[i] = l) or (abs(t[i] - l) = abs(i - c)) then ok := false;
        i := i + 1;
    end;
    Valide := ok;
end;

procedure AfficherEchiquier;
var i, j : integer;
begin
    solCount := solCount + 1;
    writeln('Solution N : ', solCount);
    for i := 1 to 8 do begin
        for j := 1 to 8 do begin
            if t[j] = i then write(' D ') else write(' . ');
        end;
        writeln;
    end;
    writeln('-----');
end;

procedure PlacerDames(c : integer);
var l : integer;
begin
    if c > 8 then AfficherEchiquier
    else begin
        for l := 1 to 8 do begin
            if Valide(l, c) then begin
                t[c] := l;
                PlacerDames(c + 1);
            end;
        end;
    end;
end;

begin
    clrscr; solCount := 0;
    PlacerDames(1);
    writeln('Nombre total de solutions : ', solCount);
    readln;
end.

```

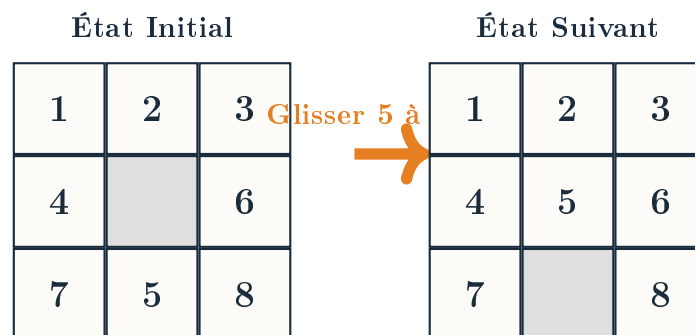
1.3 Le Jeu du Taquin (Puzzle)

1.3.1 Principe de base

Le jeu du Taquin (par exemple, le format 3×3 appelé 8-puzzle) se compose d'une grille contenant des tuiles numérotées et une case vide permettant le glissement des tuiles adjacentes. L'objectif est d'atteindre une configuration ordonnée cible (état final) à partir d'une configuration initiale désordonnée.

1.3.2 Représentation Visuelle

Voici la transition de déplacement d'une pièce (glissement vers la droite) :



1.3.3 Concepts Clés de Modélisation Informatique

Pour modéliser ce jeu, on utilise :

1. **Modélisation de l'État** : Un tableau bidimensionnel de taille 3×3 d'entiers. La valeur 0 représente la case vide.
2. **Mouvements autorisés** : Glissement de la case vide vers le haut, le bas, la gauche ou la droite. Cela implique la permutation de la case 0 avec une case adjacente valide.
3. **Arbre des configurations** : Chaque état génère jusqu'à 4 états successeurs. L'exploration exhaustive (BFS ou DFS) génère un graphe d'états qui peut atteindre $9!/2 = 181\,440$ configurations accessibles.
4. **Heuristiques d'évaluation** : Pour guider la recherche sans tout explorer, on utilise des fonctions d'estimation :
 - **Distance de Manhattan** : Somme des distances géométriques absolues de chaque case par rapport à sa position finale cible.
 - **Nombre de mal placées** : Nombre brut de tuiles qui ne sont pas à leur position correcte.

CHAPITRE 2

Le Recueil de Questions-Réponses

Voici 40 questions et réponses indispensables pour maîtriser les Algorithmes Avancés (Backtracking et Taquin) au Baccalauréat.

Question 1 : Quelle est la différence fondamentale entre une recherche naïve (brute force) et le Backtracking ?

Réponse : La brute force explore aveuglément tout l'espace de recherche, tandis que le Backtracking évalue les contraintes à chaque étape intermédiaire et abandonne immédiatement (élague) une branche dès qu'une contrainte est violée, évitant d'explorer des millions de sous-solutions inutiles.

Question 2 : Pourquoi le Backtracking est-il presque toujours implémenté sous forme récursive ?

Réponse : La récursivité gère naturellement et de manière transparente la « pile d'appels système », ce qui permet de mémoriser les choix précédents et d'effectuer des retours arrière automatiques lors du dépilement, simplifiant grandement l'écriture du code.

Question 3 : Qu'est-ce que l'« élagage » (*pruning*) dans le contexte du Backtracking ?

Réponse : C'est l'action de couper ou de ne pas explorer une branche de l'arbre de recherche dès qu'on s'aperçoit que l'état courant ne peut plus mener à une solution valide, limitant drastiquement la complexité en temps de calcul.

Question 4 : Quelle structure de données interne est manipulée lors d'une récursivité de Backtracking ?

Réponse : C'est la **pile système** (*Call Stack*) qui enregistre pour chaque appel l'état local du problème, les paramètres d'entrée et l'adresse de retour.

Question 5 : Quel est le pire cas de complexité temporelle d'un algorithme de Backtracking pour les 8 dames ?

Réponse : Le pire cas théorique reste de complexité factorielle $\mathcal{O}(N!)$ (ou exponentielle $\mathcal{O}(N^N)$), mais en pratique, l'élagage agressif réduit ce temps à une fraction de seconde sur des machines modernes.

Question 6 : Comment modélise-t-on l'état d'un échiquier de 8 dames de manière optimale à une dimension ?

Réponse : Un simple tableau de 8 entiers, `t: array[1..8] of integer`, où l'indice correspond au numéro de colonne, et la valeur `t[i]` correspond à la ligne occupée par la reine dans cette colonne.

Question 7 : Quel est l'avantage de la modélisation à une dimension par rapport à une matrice 8×8 pour les 8 dames ?

Réponse : Elle réduit la consommation mémoire, accélère les tests de validité et garantit d'emblée la contrainte technique majeure : « un seul placement possible de dame par colonne ».

Question 8 : Comment traduit-on la condition pour que deux dames placées en (L_1, C_1) et (L_2, C_2) soient sur la même ligne ou colonne ?

Réponse : Même ligne : $L_1 = L_2$. Même colonne : $C_1 = C_2$ (impossible par construction si on utilise un tableau monodimensionnel d'indices de colonnes).

Question 9 : Quelle est la formule arithmétique pour tester si deux reines en (L_1, C_1) et (L_2, C_2) sont sur une même diagonale ?

Réponse : Le test d'égalité des distances absolues : $|L_1 - L_2| = |C_1 - C_2|$.

Question 10 : Combien de solutions distinctes possède le problème classique des 8 dames ?

Réponse : Il possède exactement **92 solutions** distinctes (dont 12 solutions de base uniques sans compter les rotations et symétries).

Question 11 : Qu'est-ce que le retour sur trace automatique en programmation récursive ?

Réponse : C'est le fait que lorsque l'exécution d'une fonction récursive se termine (sans succès), le contrôle revient automatiquement à l'appelant direct à l'instruction suivant l'appel récursif, réinitialisant implicitement les variables locales.

Question 12 : Comment implémente-t-on l'annulation de choix explicitement si nécessaire ?

Réponse : Juste après l'appel récursif échoué, on réinitialise l'état modifié (par exemple, remettre la case de l'échiquier à vide ou à **false**) avant de passer à l'itération de boucle suivante.

Question 13 : Quel type de problème se prête particulièrement bien au Backtracking ?

Réponse : Les problèmes de décision (est-ce possible ?), de dénombrement (combien de solutions ?), et d'optimisation sous contraintes (quelle est la meilleure solution ?).

Question 14 : Qu'est-ce que le problème du Taquin ?

Réponse : C'est un jeu de réflexion constitué d'une grille de tuiles numérotées glissant dans un cadre, l'une d'elles étant vide. Le but est de remettre la grille en ordre en faisant glisser les tuiles.

Question 15 : Comment modélise-t-on informatiquement une grille de Taquin 3×3 ?

Réponse : Par une matrice bidimensionnelle d'entiers `grille: array[1..3, 1..3] of integer`, où le chiffre 0 désigne la case vide (le trou).

Question 16 : Quels sont les 4 mouvements physiques possibles de la case vide dans le Taquin ?

Réponse : Haut (Up), Bas (Down), Gauche (Left) et Droite (Right), impliquant chacun une permutation de coordonnées valide dans la matrice.

Question 17 : Pourquoi dit-on que le Taquin génère un graphe d'états plutôt qu'un arbre pur ?

Réponse : Parce qu'un état (configuration de grille) peut être atteint par plusieurs chemins différents (successions de mouvements), créant des cycles que l'algorithme doit détecter pour éviter les boucles infinies.

Question 18 : Qu'est-ce qu'une « fonction heuristique » dans la résolution du jeu du Taquin ?

Réponse : C'est une fonction qui évalue la distance d'un état intermédiaire par rapport à l'état final cible, permettant de guider l'exploration vers les branches les plus prometteuses.

Question 19 : Qu'est-ce que la « Distance de Manhattan » pour une tuile du Taquin ?

Réponse : C'est la somme des distances absolues en lignes et en colonnes séparant la position actuelle de la tuile de sa position correcte finale : $d = |x_1 - x_2| + |y_1 - y_2|$.

Question 20 : Pourquoi la distance de Manhattan est-elle jugée plus performante que le simple calcul de tuiles mal placées ?

Réponse : Elle prend en compte l'éloignement physique réel de chaque tuile par rapport à sa cible, fournissant une estimation numérique beaucoup plus fine de l'effort restant à fournir.

Question 21 : Comment le Backtracking évite-t-il les boucles infinies dans un graphe d'états comme celui du Taquin ?

Réponse : En stockant la liste des configurations déjà explorées (liste des « États Visités ») et en interdisant de ré-explorer une configuration déjà présente dans cette liste.

Question 22 : Qu'est-ce que la condition d'arrêt d'une recherche récursive de Backtracking ?

Réponse : Soit la découverte d'une solution valide complète (ex: colonne > 8 pour les dames), soit l'épuisement de toutes les options valides d'une branche sans solution.

Question 23 : Quelle est la différence entre une recherche en profondeur (DFS) et une recherche en largeur (BFS) ?

Réponse : Le DFS s'enfonce au maximum le long d'une seule branche avant de faire demi-tour (idéal pour le Backtracking récursif), tandis que le BFS explore niveau par niveau (recherche la solution en un minimum de mouvements).

Question 24 : Pourquoi l'utilisation de variables globales dans un Backtracking récursif peut-elle être risquée ?

Réponse : Si les variables globales stockent l'état local d'un niveau récursif sans être correctement restaurées, les appels parents liront des données corrompues créées par les appels enfants.

Question 25 : Quel est le danger d'une absence de condition d'arrêt valide dans un Backtracking récursif ?

Réponse : Cela provoque un appel récursif infini, menant rapidement à une erreur d'exécution système fatale par **débordement de pile** (*Stack Overflow*).

Question 26 : Qu'est-ce que le problème du voyageur de commerce (*TSP*) et comment s'applique-t-il au Backtracking ?

Réponse : C'est la recherche du plus court chemin passant par un ensemble de villes. Le Backtracking peut trouver le chemin optimal en explorant tous les itinéraires valides tout en éliminant les chemins qui dépassent déjà le minimum courant trouvé.

Question 27 : Que signifie le terme « espace d'états » d'un problème ?

Réponse : C'est l'ensemble de toutes les configurations de variables ou d'états qu'il est physiquement ou mathématiquement possible de former dans le cadre du problème, qu'elles soient valides ou non.

Question 28 : Pourquoi le tri préalable de nos choix dans un Backtracking peut-il accélérer l'exécution ?

Réponse : En explorant d'abord les choix les plus prometteurs (heuristique de choix), on trouve une solution valide plus vite, permettant d'élaguer plus massivement les branches suivantes par comparaison.

Question 29 : Comment détecte-t-on si une configuration de Taquin est non résoluble ?

Réponse : En calculant le nombre d'**inversions** dans l'ordre des chiffres. Si la parité du nombre d'inversions ne correspond pas à celle de la grille cible, la configuration ne pourra jamais être résolue.

Question 30 : Quel est le rôle d'un pivot ou d'un état racine dans le graphe de recherche ?

Réponse : C'est le point d'entrée unique de l'algorithme, représentant la situation initiale désordonnée du problème à partir de laquelle s'embranchent toutes les décisions de recherche.

Question 31 : Comment code-t-on le retour d'une fonction récursive qui doit s'arrêter dès qu'UNE SEULE solution est trouvée ?

Réponse : On utilise une fonction booléenne. Si un appel enfant retourne **true**, l'appel parent cesse d'explorer la boucle et propage immédiatement **true** vers le haut.

Question 32 : Quelle est la différence entre une heuristique admissible et non admissible ?

Réponse : Une heuristique admissible ne surestime jamais le coût réel restant pour atteindre la cible (ex: Manhattan), garantissant de trouver la solution optimale.

Question 33 : Qu'est-ce que l'algorithme A* (A-Star) et son lien avec le Taquin ?

Réponse : C'est un algorithme de recherche de chemin qui combine le coût réel accumulé $g(n)$ et le coût heuristique estimé $h(n)$ pour trouver la solution optimale de déplacement au Taquin de manière extrêmement efficace.

Question 34 : Comment implémente-t-on un test de collision de diagonale sans utiliser la fonction **Abs** (valeur absolue) ?

Réponse : En testant séparément les deux conditions diagonales : $(L_1 - C_1 = L_2 - C_2)$ et $(L_1 + C_1 = L_2 + C_2)$.

Question 35 : Quel type de boucle est utilisé pour tester les 8 lignes possibles dans la procédure des 8 dames ?

Réponse : Une boucle **Pour** allant de 1 à 8, qui s'exécute de façon indépendante dans chaque niveau d'activation de la récursion de colonne.

Question 36 : Pourquoi le nombre de cases d'un échiquier 8×8 est un faux problème de stockage pour le Backtracking ?

Réponse : Car l'algorithme n'a pas besoin de stocker l'historique complet de tous les essais ratés, mais uniquement le chemin de décisions courant représenté par les appels actifs.

Question 37 : Dans le problème des 8 dames, pourquoi n'y a-t-il pas besoin d'un tableau pour suivre les colonnes déjà occupées ?

Réponse : Parce que la variable récursive c (colonne) augmente de $+1$ à chaque étape, garantissant de fait qu'on ne traite qu'une dame par colonne.

Question 38 : Combien d'états uniques possède un 15-puzzle (Taquin 4×4) ?

Réponse : Il possède $16! \approx 20,9 \times 10^{12}$ états possibles, dont la moitié (environ 10^{13}) sont résolubles, ce qui rend l'heuristique indispensable.

Question 39 : Quelle est la complexité spatiale d'un parcours de Backtracking à N niveaux d'arbre ?

Réponse : Elle est linéaire $\mathcal{O}(N)$, car on ne conserve en mémoire que la profondeur du chemin exploré courant sur la pile d'appels.

Question 40 : Pourquoi le site d'excellence **mathbacai.com** insiste-t-il sur la notation pointée pour manipuler les états ?

Réponse : Pour assurer une écriture syntaxique claire et normalisée des types de données personnalisés (Record Pascal) modélisant les structures de grilles et de coordonnées d'un problème.

CHAPITRE 3

Quiz d'Évaluation & Fiches Mémo

3.1 Quiz d'Évaluation (10 Questions Types Bac)

1. Le problème des 8 dames résolu par Backtracking explore 8^8 positions dans le pire des cas.

Réponse : FAUX.

Justification : Grâce à l'élagage, le Backtracking n'explore qu'une fraction infime de ces positions. Dès que deux dames se menacent au niveau de la ligne ou des diagonales, la recherche dans cette branche est immédiatement stoppée et l'algorithme recule.

2. La notation pointée est indispensable pour modifier le contenu d'un champ d'un enregistrement en Pascal.

Réponse : VRAI (ou via la structure alternative).

Justification : En Pascal, pour modifier un champ `nom` d'une variable d'enregistrement `E`, on écrit `E.nom := '...'`. On peut également utiliser le raccourci syntaxique `with E do nom := '...'`.

3. La distance de Manhattan calcule la somme des distances en diagonale entre une tuile du Taquin et sa destination.

Réponse : FAUX.

Justification : La distance de Manhattan n'autorise que les mouvements rectilignes (horizontaux et verticaux). Elle se calcule par $|x_1 - x_2| + |y_1 - y_2|$ et exclut les déplacements diagonaux.

4. Le Backtracking utilise une file d'attente (FIFO) pour stocker les choix en cours.

Réponse : FAUX.

Justification : Le Backtracking repose sur un parcours en profondeur et utilise une **pile** (LIFO - Last In First Out), matérialisée par la pile d'appels système lors de la récursion.

5. Dans l'implémentation récursive des 8 dames, la procédure doit obligatoirement être une fonction booléenne.

Réponse : FAUX.

Justification : Si on veut afficher **toutes** les solutions possibles, une procédure récursive simple (`procedure PlacerDames(c: integer)`) est idéale. La fonction booléenne n'est requise que si l'on souhaite s'arrêter dès la découverte de la première solution valide.

6. Dans le Taquin, un état peut être représenté par un tableau unidimensionnel de taille 9.

Réponse : VRAI.

Justification : Bien que conceptuellement bidimensionnel (3×3), un tableau de 1 à 9 peut modéliser le Taquin pour optimiser les calculs de parité d'inversions ou simplifier la mémoire.

7. Un élagage inefficace ou absent dans un Backtracking conduit à une erreur d'exécution.

Réponse : FAUX.

Justification : Cela ne conduit pas à une erreur d'exécution, mais à une explosion du temps de calcul (qui devient extrêmement long). L'erreur d'exécution (débordement de pile) survient uniquement en cas d'absence de condition d'arrêt (récursion infinie).

8. Le nombre total de configurations valides et résolubles pour un 8-puzzle (Taquin 3×3) est de 181 440.

Réponse : VRAI.

Justification : Le nombre total de permutations de 9 éléments est $9! = 362\,880$. Or, seulement la moitié de ces configurations sont physiquement résolubles, soit $9!/2 = 181\,440$ états.

9. L'expression $\text{abs}(t[i] - 1) = \text{abs}(i - c)$ vérifie si deux dames sont sur une même ligne.

Réponse : FAUX.

Justification : Cette formule vérifie si elles se trouvent sur une **même diagonale**. L'alignement sur une même ligne est vérifié par le test simple $t[i] = 1$.

10. Pour un Taquin, l'heuristique des "tuiles mal placées" fournit une estimation toujours inférieure ou égale au nombre réel de déplacements requis.

Réponse : VRAI.

Justification : Comme chaque tuile mal placée nécessite au minimum un déplacement pour retourner à sa case cible, le nombre de tuiles mal placées est toujours inférieur ou égal au nombre réel d'étapes nécessaires. C'est une heuristique admissible.

3.2 Le Tableau Récapitulatif d'Arborescence

Algorithme	Espace de Recherche (Naïf)	Complexité Spatiale	Approche de Guidage
8 Dames	$64^8 \approx 2,8 \times 10^{14}$	$\mathcal{O}(N)$	Élagage par contraintes
Taquin	$9! = 362\,880$ (8-puzzle)	$\mathcal{O}(N)$ (avec Visités)	Distance de Manhattan
Brute Force	Dépendant du problème	$\mathcal{O}(1)$ ou $\mathcal{O}(N)$	Exploration aveugle

Table 3.1: Synthèse comparative des concepts d'Algorithmes Avancés

3.3 Fiche Mémo : Les 5 Règles d'Or pour le Bac

Les 5 Règles d'Or d'Évaluation de mathbacai.com

1. **Toujours identifier la structure de décision** : Pour le Backtracking, assurez-vous de bien définir ce que représente chaque niveau de récursion (ex: colonne) et l'ensemble des choix possibles à chaque niveau (ex: les 8 lignes d'échecs).
2. **Ne négligez jamais l'annulation du choix (Backtrack)** : Si vous modifiez un état global partagé par d'autres appels récurifs, veillez à toujours réinitialiser cette valeur après l'échec de l'appel enfant.
3. **Blindez la condition d'arrêt** : Une fonction récursive sans condition d'arrêt rigoureuse ou mal formulée conduira instantanément à l'arrêt du programme par **Stack Overflow** (Débordement de pile).
4. **Validez la parité d'une grille de Taquin** : Avant d'entamer une recherche lourde, déterminez si le Taquin est mathématiquement résoluble en vérifiant la cohérence du nombre d'inversions.
5. **Pensez à l'élagage hâtif** : Mettez les conditions de vérification de contraintes au tout début de vos structures décisionnelles pour éliminer au plus tôt les calculs inutiles.